

Summary of Changes Made - Networking

Peer Team Feedback

After receiving feedback from other teams during the integration review, two key features were identified as missing from our current system: a **Turn Timer** and a **Content Filter System**. Both were essential to improving fairness, gameplay flow, and creating a safer environment for players. We've now implemented both features and ensured they integrate cleanly into our existing system architecture.

1. Turn Timer Functionality

The feedback pointed out the lack of a mechanism to handle players who go idle or take too long to make a move. To address this, we implemented a *TurnTimer* class that monitors each player's turn duration. The timer starts at the beginning of a player's turn and counts down. If the player fails to act within the allowed time, the system flags a timeout and simulates disconnection from the game session.

In addition to managing timeout scenarios, the timer includes the ability to:

- **Pause** during unexpected issues like disconnection.
- **Resume** when the player reconnects.
- **Reset** upon successful turn completion.
- **Notify** the player when only 10 seconds remain on the clock.

This change improves the pacing of the game and prevents stalling, which is especially important in a real-time or competitive setting.

2. Content Filter System

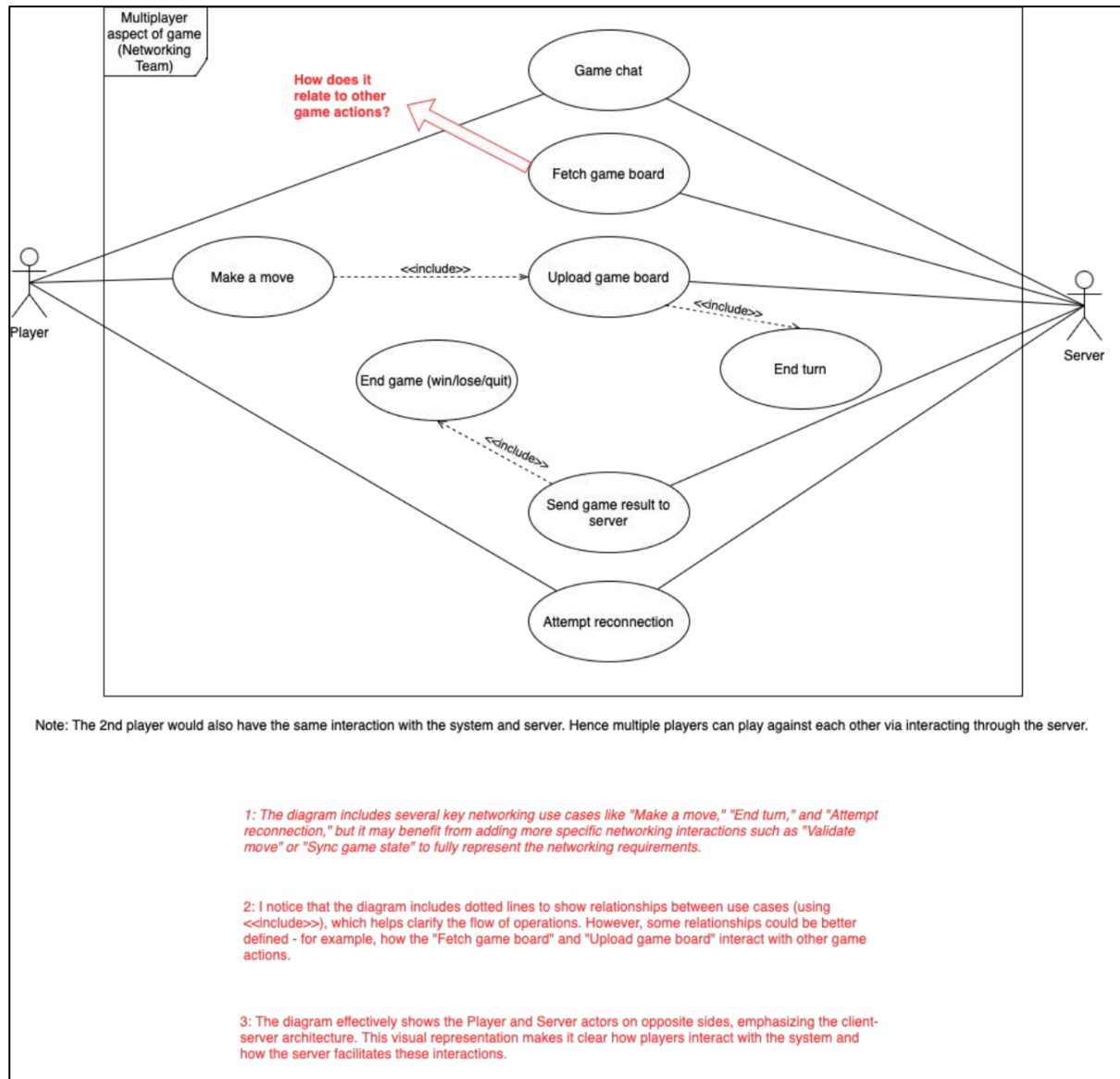
Another suggestion was to handle inappropriate language in the in-game chat. As a solution, we extended the *ChatManager* to include a content filter that checks all incoming messages against a curated list of offensive words. If a message contains any of these terms, it is automatically rejected and not added to the chat history. The player is warned via a console message.

This ensures the chat system promotes respectful communication and discourages toxic behavior. The list of inappropriate words was sourced from an open-source project and adapted to cover a wide range of offensive terms.

Although reporting functionality is not yet fully implemented, the system can easily be extended to flag users or notify moderators when inappropriate messages are blocked.

Use Case Diagram:

Following is the annotated diagram received from Iteration 2 feedback:



Updated Diagram:

Addressing feedback comment 1: In our project, move validation is done by Game Logic Team and occurs when “Make a move” use case is called (added as a note in updated diagram). “Sync game state” is performed by “Fetch game board” and “Upload game board” use cases.

Addressing feedback comment 2: Added an <<include>> relation from “Fetch game board” to “Make a move” to show that a move is only made after updated game board is fetched from server. Similarly, “upload game board” occurs after a move is made and then the turn ends.

Addressing feedback from the other (2nd) team: Added an extend relation from “Make a move” to “End game”, so that when a player does not make a move, the game automatically ends.

